

Содержание

Введение.....	4
1. Цель и задачи проекта.....	5
2. Характеристика объекта автоматизации.....	6
3. Классы и характеристики пользователей.....	9
4. Обзор аналогов.....	10
4.1. Сравнительная таблица аналогов.....	10
4.2. Анализ актуальности.....	11
4.3. Анализ требований.....	11
4.4. Выводы по аналогам.....	12
5. Требования к функциональным характеристикам.....	13
6. Технические требования.	23
6.1. Требования к программному обеспечению.....	23
6.2. Требования к организационному обеспечению.....	24
7. Оценка трудоемкости разработки ПО на основе вариантов использования	26
7.1. Определение весовых показателей действующих лиц.....	26
7.2. Определение весовых показателей вариантов использования.....	26
7.3. Подсчет количества вариантов использования каждого типа.....	26
7.4. Определение технической сложности проекта.....	27
7.5. Определение уровня квалификации разработчиков.....	28
7.6. Оценка трудоемкости проекта.....	28
8. Информационное обеспечение системы	29
9. Метрика спроектированной системы.....	35
9.1. Описание классов.....	35
9.2. Расчёт метрики.....	37
10. Руководство пользователя.....	40
11. Структура приложения.....	43
11.1. Архитектура приложения.....	43
11.2. Листинг.....	44
12. Тестирование.....	55
12.1. Структурное тестирование.....	55
12.2. Функциональное тестирование.....	60
13. Заключение.....	63
14. Список использованных источников.....	64

Введение

Современные образовательные процессы требуют всё большего уровня автоматизации. Особенно это касается таких рутинных операций, как учёт успеваемости студентов, где важны точность, наглядность и скорость обработки данных.

Разработка программного обеспечения для хранения и анализа оценок студентов позволяет частично или полностью исключить ручную обработку информации, снизить вероятность ошибок, повысить эффективность работы преподавателя и упростить взаимодействие с данными.

В рамках курсовой работы необходимо разработать систему, состоящую из нескольких форм, реализующих базовые функции учёта успеваемости:

- Выбор группы и предмета,
- Просмотр списка студентов,
- Выставление и редактирование оценок,
- Расчёт среднего балла за полугодия и год.

1. Цель и задачи проекта

Цель проекта:

Разработать информационную систему учёта успеваемости студентов позволяющую сократить временные затраты преподавателя на обработку информации.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Разработать требования к функциональным, техническим характеристикам модуля информационной системы учета успеваемости студентов.
2. Провести анализ существующих аналогичных решений, предлагающих функции автоматизации учёта успеваемости.
3. Провести анализ требований, составить план по реализации модуля.
4. Провести оценку трудозатрат.
5. Выполнить реализацию модуля информационной системы учета успеваемости студентов.
6. Провести тестирование системы.
7. Составить рекомендации по дальнейшей доработке системы.

2. Характеристика объекта автоматизации

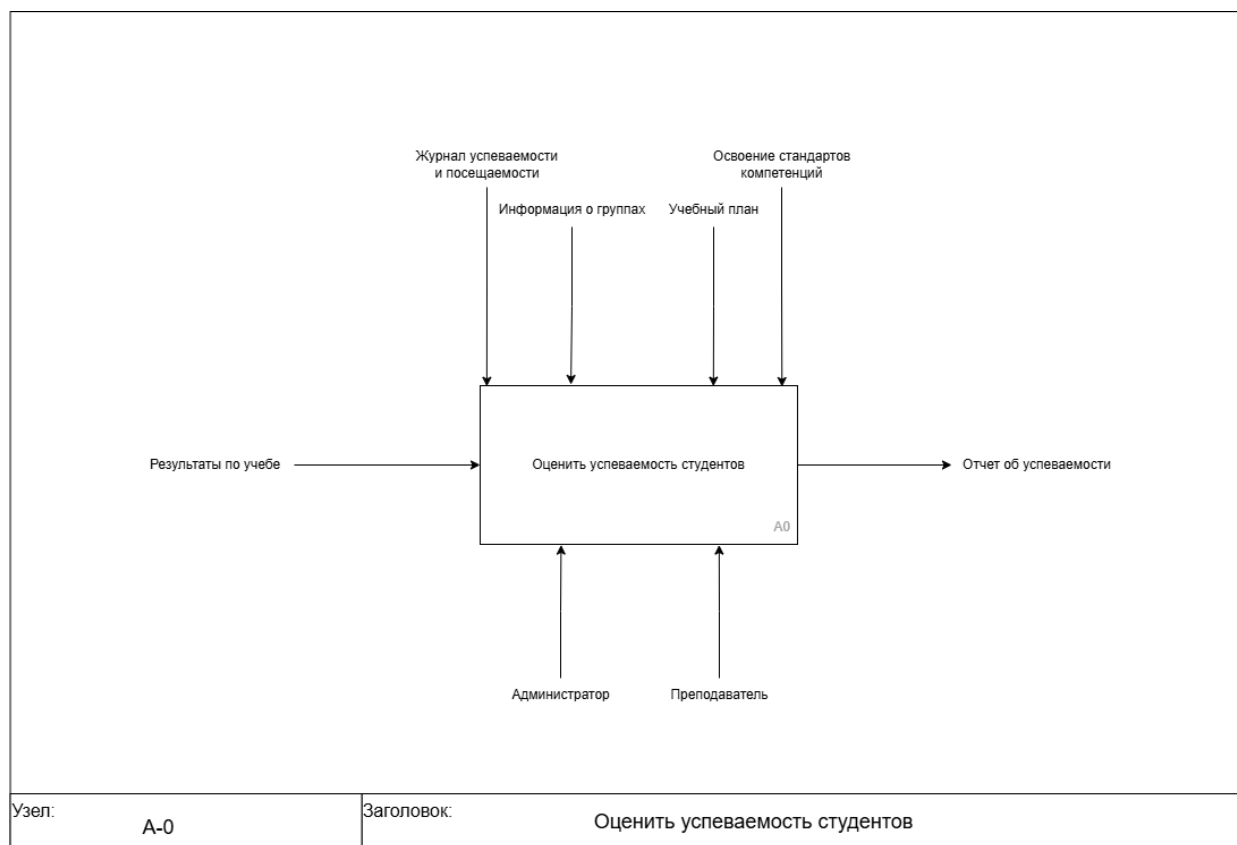


Рис. 1. Узел A-0 диаграммы IDEF0. Модуль информационной системы

Программа учёта успеваемости студентов разрабатывается для образовательного учреждения. Она предназначена для преподавателей и администраторов, которые работают с программой на своих рабочих местах. Система позволяет оценивать успеваемость студентов, просматривать результаты по учебе и формировать отчёты. Это представлено на рисунке 1.

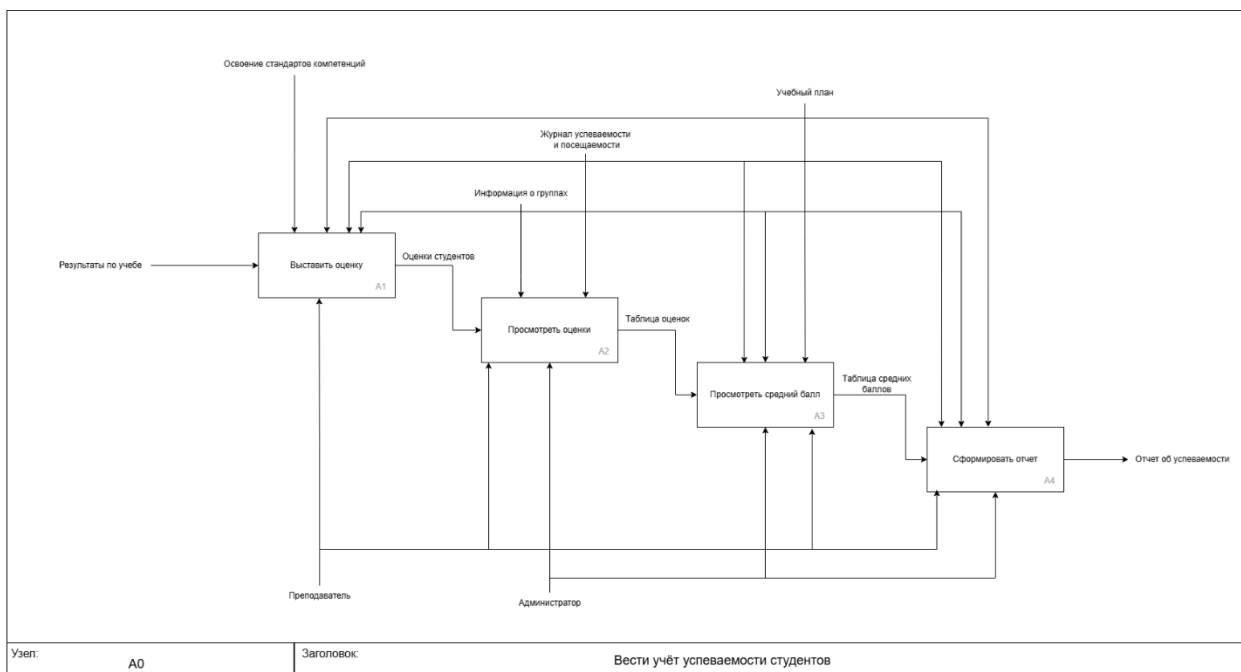


Рис. 2. Узел A0 диаграммы IDEF0. Вести учёт успеваемости студентов

Преподаватель начинает процесс с выбора группы, предмета и даты, после чего просматривает данные о студентах и вносит соответствующие оценки. Затем система позволяет как преподавателю, так и администратору увидеть текущие результаты обучения. Далее, на основе внесённых данных, программа автоматически рассчитывает средний балл за год, что помогает оценить общую успеваемость каждого ученика. В завершение администратор имеет возможность сформировать отчёт, содержащий детальную информацию о результатах обучения всех студентов. Все эти процессы наглядно представлены на рисунке 2, где показано взаимодействие между пользователями и основными функциями системы.

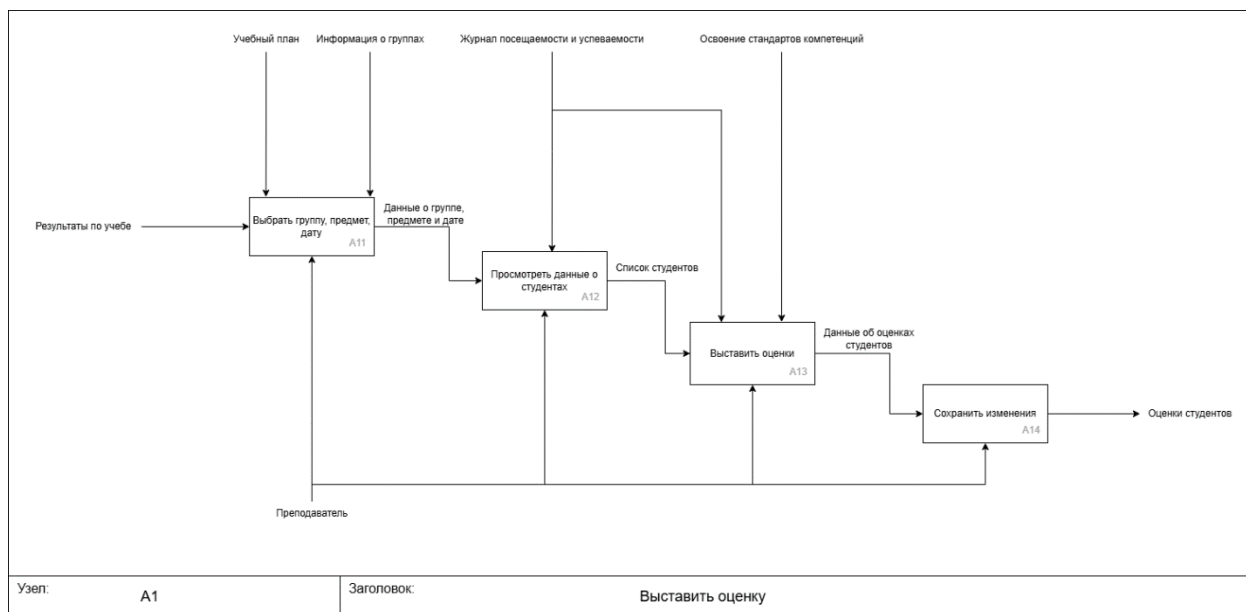


Рис. 3. Узел A1 диаграммы IDEF0. Выставить оценку

Преподаватель начинает процесс с выбора группы, предмета и даты, после чего система предоставляет список студентов. Далее он может просмотреть данные о каждом студенте, включая их успеваемость и посещаемость, а затем внести новые оценки. Изменения сохраняются в базе данных, что обеспечивает актуальность информации о результатах обучения. Этот процесс основан на данных из учебного плана, информации о группах и журнала посещаемости, а результат - обновлённые оценки студентов, они используются для формирования отчётов и анализа успеваемости. Все этапы взаимосвязаны и обеспечивают целостность системы управления оценками.

3. Классы и характеристики пользователей

В системе предусмотрены три класса пользователей:

1. Преподаватель — лицо, ответственное за выставление оценок студентам и управление учебным процессом.
2. Администратор — лицо, осуществляющее общее управление системой, включая настройку параметров и формирование отчетов.
3. Студент — пользователь, который может просматривать свои оценки и результаты обучения.

Таблица 1.

Пользователь	Доступ	Обязанности	Навыки
Преподаватель	Доступ к формам выставления оценок и просмотра данных	Выставление оценок студентам.	Знание учебного плана, умение работать с базой данных, педагогический опыт.
Администратор	Полный доступ ко всем функциям системы	Управление доступом, формирование отчетов, мониторинг данных.	Администрирование БД, аналитические навыки, знание ГОСТов и стандартов.
Студент	Ограниченный доступ к просмотру данных	Просмотр своих оценок и результатов обучения.	Основные компьютерные навыки, умение интерпретировать данные.

4. Обзор аналогов

4.1. Сравнительная таблица налогов

Для анализа существующих программных решений, ориентированных на автоматизацию учета успеваемости студентов, были проанализированы несколько популярных платформ. В таблице ниже представлены ключевые функции, реализованные в выбранных системах, а также их соответствие требованиям проекта.

Таблица 2.

Характеристика	Сетевой Город	Moodle	Excel/Google Таблицы	Моя программа
Локальное использование без интернета	—	—	+	+
Простой интерфейс	—	—	+	+
Наличие автоматического расчёта средних баллов	+	+	—	+
Возможность редактирования оценок	+	+	+	+
Централизованное хранение данных	+	+	—	+(PostgreSQL)
Минимальные системные требования	—	—	+	+
Удобство для преподавателя	+	+	+	+

Открытость исходного кода	—	+	—	+ (локальная реализация)
Бесплатное использование	+	+	+	+
Поддержка работы с базой данных	+	+	—	+(PostgreSQL)

4.2. Анализ актуальности

На современном рынке существует множество решений для автоматизации учёта успеваемости студентов. Важно отметить, что большинство систем предлагают схожие функции, такие как выставление оценок, просмотр данных и формирование отчётов. Однако некоторые системы могут иметь ограничения в поддержке расчёта среднего балла, генерации детализированных отчётов и уведомления пользователей о изменениях.

4.3. Анализ требований

При выборе программного обеспечения для автоматизации учёта успеваемости студентов следует учитывать следующие требования:

Функциональность:

- Система должна включать все необходимые функции для ведения учёта оценок, анализа успеваемости и формирования статистики. Это включает возможность выставления оценок, просмотра данных студентов, расчёта среднего балла за год и генерации отчётов.

Интеграция:

- Программа должна быть способна интегрироваться с существующими образовательными платформами и базами данных, а также поддерживать настройку под специфические нужды учебного заведения.

Пользовательский интерфейс:

- Интерфейс должен быть удобным и интуитивно понятным для всех категорий пользователей - преподавателей, администраторов и студентов.

Безопасность:

- Важно обеспечить защиту данных студентов и информации о результатах обучения, включая контроль доступа и шифрование передаваемых данных.

4.4. Выводы по аналогам

Анализ показал, что большинство существующих решений либо слишком сложны для ежедневного использования, либо не обеспечивают надёжного хранения данных.

Программа функционирует в локальном режиме без необходимости подключения к интернету, не требует установки и настройки сервера, имеет интуитивно понятный интерфейс, обеспечивает хранение данных в базе данных, реализует автоматический расчёт средних баллов и характеризуется простотой освоения и использования.

Таким образом, разработка является актуальной альтернативой тем, кто хочет быстро и надёжно учитывать успеваемость студентов без избыточных функций и внешних зависимостей.

5. Требования к функциональным характеристикам

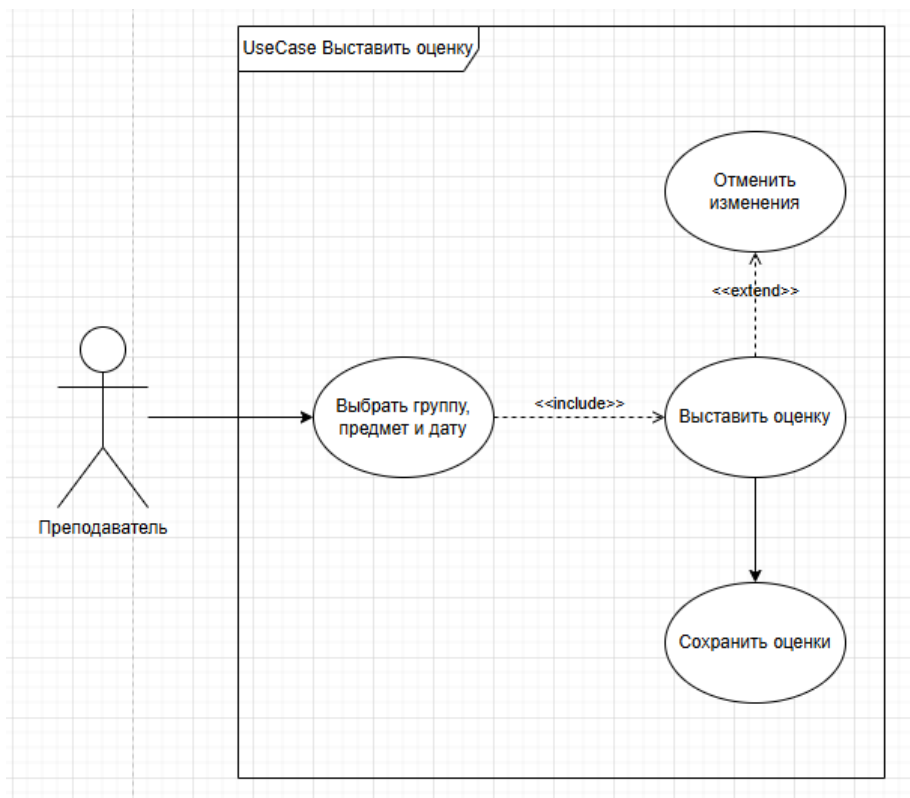


Рис. 4. Диаграмма UseCase. Выставить оценку

Идентификатор и название варианта использования	UC-1. Выставить оценку.
Автор	Травкина Ксения
Дата создания:	28.04.2025
Основное действующее лицо: Преподаватель	Дополнительное действующее лицо: Система StudentsGradesApp
Описание	Преподаватель вводит или редактирует оценки студентам за выбранный предмет и дату занятия.
Условие-триггер	Преподаватель выражает намерение выставить оценки студентам.
Предварительные условия	PRE-1: Преподаватель авторизован в системе. PRE-2: Параметры (группа, предмет, дата) успешно выбраны. PRE-3: Существует список студентов для выбранной группы и предмета. PRE-4: Система готова к вводу данных.

Идентификатор и название варианта использования	UC-1. Выставить оценку.
Выходные условия	POST-1: Оценки студентов сохранены в базе данных системы StudentsGradesApp. POST-2: Список оценок обновлен с учетом новых данных. POST-3: Система готова к дальнейшей работе POST-4: Преподаватель может вернуться к предыдущему шагу или продолжить работу.
Нормальное направление	1.0. Основной сценарий выставления оценок 1. Преподаватель запрашивает форму ввода оценок. 2. Система загружает список студентов и текущие оценки по выбранному предмету и дате. 3. Преподаватель выбирает студента из списка. 4. Преподаватель вводит новую оценку или изменяет существующую. 5. Преподаватель повторяет шаги 3–4 для всех студентов в группе. 6. Преподаватель нажимает кнопку «Сохранить». 7. Система проверяет корректность введенных данных. 8. Система сохраняет изменения в базе данных.
Альтернативные направления	1.1. Отменить ввод оценок 1. Преподаватель решает отменить внесённые изменения до сохранения. 2. Система откатывает данные к предыдущему состоянию. Преподаватель возвращается к форме выбора параметров.
Исключения	1.0.E1 Ошибка при сохранении данных 1. Система не может сохранить оценки (например, из-за ошибки подключения к БД). 2. Система выводит сообщение об ошибке. 3. Преподаватель может попробовать сохранить данные снова или отменить ввод.
Приоритет	Высокий
Частота использования	Приблизительно 100 пользователей в день. Пиковая нагрузка - с 8:00 до 17:00 местного времени.
Бизнес-правила	BR-5, BR-7, BR-12, BR-15

Идентификатор и название варианта использования	УС-1. Выставить оценку.
Другая информация	1. Преподаватель должен иметь возможность изменять введённые оценки до их сохранения. 2. Система должна позволять преподавателю просматривать историю своих последних 10 вводов оценок.
Предположения	Предполагается, что 25% преподавателей будут использовать данную функцию для массового ввода оценок (источник: внутренняя статистика системы).

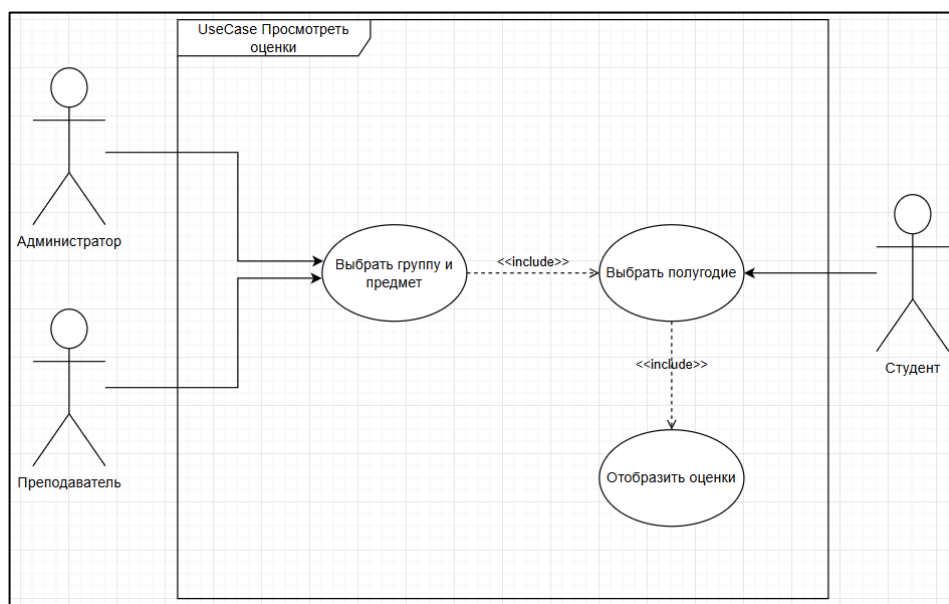


Рис 5. Диаграмма UseCase: просмотреть оценки

Идентификатор и название варианта использования	УС-2. Просмотреть оценки.
Автор	Травкина Ксения
Основное действующее лицо: Преподаватель	Дополнительное действующее лицо: Студент, Администратор
Описание	Данный кейс описывает процесс просмотра текущих оценок студентов по выбранному предмету и группе.
Условие-триггер	Желание преподавателя просмотреть оценки студентов за конкретный предмет и группу.

Идентификатор и название варианта использования	UC-2. Просмотреть оценки.
Предварительные условия	PRE-1: Преподаватель авторизован в системе StudentsGradesApp. PRE-2: База данных содержит актуальные данные о студентах, предметах и группах. PRE-3: Система готова к работе с данными.
Выходные условия	POST-1: Данные оценок отображены на экране системы. POST-2: Преподаватель может вернуться к предыдущему шагу или продолжить работу.
Нормальное направление	1.0. Основной сценарий просмотра оценок 1. Преподаватель открывает форму выбора параметров. 2. Преподаватель выбирает группу и предмет. 3. Преподаватель (по желанию) выбирает полугодие. 4. Система загружает оценки студентов из базы данных. 5. Система отображает таблицу с оценками. 6. Преподаватель может фильтровать или сортировать данные. 7. Преподаватель завершает просмотр.
Альтернативные направления	1.1. Изменить параметры просмотра 1. Преподаватель изменяет группу, предмет или полугодие. 2. Система перезагружает данные. 3. Возврат к шагу 4 основного сценария. 1.2. Пользователь не найден 1. Система проверяет права доступа. 2. Если у студента или администратора нет прав - выводится сообщение "Доступ запрещён". 3. Кейс завершается.
Исключения	1.0.E1 Нет данных для просмотра 1. Система не находит записей оценок для указанного предмета и группы. 2. Система выводит сообщение: "Нет данных об оценках за выбранный период". 3. Преподаватель может вернуться к выбору других параметров.

Идентификатор и название варианта использования	UC-2. Просмотреть оценки.
Исключения	1.0.E2 Ошибка подключения к БД 1. Система не может загрузить данные из-за ошибки базы данных. 2. Система выводит сообщение: "Ошибка подключения к базе данных". 3. Преподаватель может попробовать снова.
Приоритет	Высокий
Частота использования	Более 100 пользователей в день. Пиковая нагрузка - с 14:00 до 16:00 местного времени.
Бизнес-правила	BR-5, BR-7, BR-12, BR-15
Другая информация	1. Преподаватель должен иметь возможность сохранить текущий запрос и использовать его повторно. 2. Система должна отображать историю просмотров (последние 10 запросов). 3. У студента должна быть возможность подписаться на уведомления об изменениях в оценках (в будущих версиях).
Предположения	Предполагается, что 30% преподавателей будут использовать данную функцию ежедневно; Студенты будут чаще запрашивать оценки в предэкзаменационный период. (Источник: тестирование системы в течение двух месяцев)

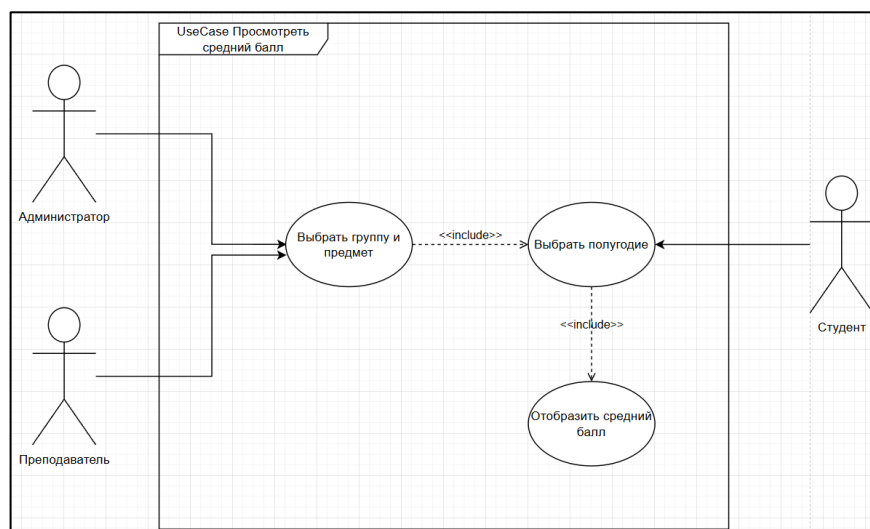


Рис 6. Диаграмма UseCase: просмотреть средний балл

Идентификатор и название варианта использования	УС-3. Просмотреть средний балл.
Автор Травкина Ксения	Дата создания: 28.04.2025
Основное действующее лицо: Преподаватель	Дополнительное действующее лицо: Студент, Администратор
Описание	Данный кейс описывает процесс получения информации о среднем балле студентов по выбранному предмету и группе. Этот кейс позволяет преподавателю анализировать успеваемость в разрезе группы, а также дает возможность студенту видеть свой личный средний балл.
Условие-триггер	Желание преподавателя, студента или администратора просмотреть средний балл студентов по выбранному предмету.
Предварительные условия	PRE-1: Преподаватель, студент или администратор авторизован(а) в системе StudentsGradesApp. PRE-2: Существует список оценок за выбранный период. PRE-3: Система готова к выводу аналитических данных.
Выходные условия	POST-1: Средний балл успешно рассчитан и отображён на экране. POST-2: Преподаватель, студент или администратор может продолжить работу или завершить текущий кейс.
Нормальное направление	1.0. Основной сценарий просмотра среднего балла 1. Преподаватель открывает форму выбора параметров. 2. Преподаватель выбирает группу и предмет. 3. Преподаватель (по желанию) указывает полугодие. 4. Система загружает данные оценок студентов. 5. Система рассчитывает средний балл по группе или отдельному студенту. 6. Система отображает результаты расчёта среднего балла. 7. Преподаватель может экспортировать данные или вернуться к другим функциям. <i>[Примечание: для студента шаги 1–2 заменяются автоматическим определением группы и предмета, основываясь на его учетной записи.]</i>

Идентификатор и название варианта использования	УС-3. Просмотреть средний балл.
Альтернативные направления	1.1. Изменить параметры расчёта 1. Преподаватель изменяет группу, предмет или полугодие. 2. Система пересчитывает средний балл. 3. Возврат к шагу 4 нормального поведения. 1.2. Нет данных для расчёта 1. Система не находит оценок для указанного периода. 2. Система выводит сообщение: "Нет данных для расчёта среднего балла". 3. Преподаватель может выбрать другие параметры или завершить кейс.
Исключения	1.0.E1 Ошибка при расчёте среднего балла 1. Система не может рассчитать средний балл из-за внутренней ошибки. 2. Система выводит сообщение: "Ошибка расчёта среднего балла". 3. Преподаватель может повторить попытку.
Приоритет	Средний
Частота использования	Около 50 обращений в день. Наибольшая активность наблюдается в конце семестра перед выставлением итоговых оценок.
Бизнес-правила	BR-8, BR-10, BR-13, BR-16
Другая информация	1. Преподаватель должен иметь возможность сохранить текущий расчет и использовать его в будущем. 2. Система должна предоставлять график средних баллов (в будущих версиях). Расчёт среднего балла должен учитывать все оценки за указанный период без исключений.
Предположения	Предполагается, что 40% преподавателей будут использовать этот кейс еженедельно. Студенты чаще всего будут запрашивать средний балл в предэкзаменационный период. (Источник: тестирование системы в учебной группе за последние 3 месяца.)

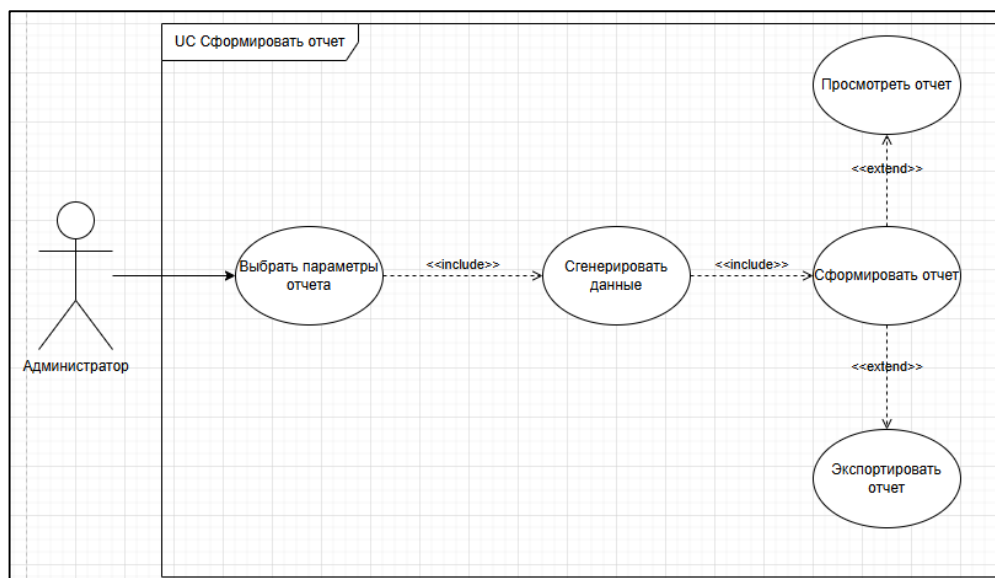


Рис 7. Диаграмма UseCase: сформировать отчет

Идентификатор и название варианта использования	UC-4. Сформировать отчет.
Автор	Травкина Ксения
Дата создания:	28.04.2025
Основное действующее лицо: Администратор	Дополнительное действующее лицо: Система StudentsGradesApp
Описание	Данный кейс описывает процесс создания аналитического отчёта об успеваемости студентов по выбранным параметрам. Этот кейс позволяет администратору анализировать общую успеваемость студентов и генерировать отчёты для административных целей.
Условие-триггер	Желание администратора сформировать отчёт об успеваемости студентов.
Предварительные условия	PRE-1: Администратор авторизован в системе StudentsGradesApp. PRE-2: Параметры отчёта (группа, предмет, полугодие) успешно выбраны. PRE-3: База данных содержит актуальные данные об оценках студентов. PRE-4: Система готова к генерации отчётов.
Выходные условия	POST-1: Отчёт сформирован и готов к экспорту. POST-2: Данные отчёта сохранены в системе. POST-3: Администратор может просмотреть или экспортировать отчёт. POST-4: Система готова к следующему взаимодействию.

Идентификатор и название варианта использования	UC-4. Сформировать отчет.
Нормальное направление	1.0. Основной сценарий формирования отчета 1. Администратор открывает форму выбора параметров. 2. Администратор выбирает группу, учебный предмет и полугодие. 3. После выбора параметров система загружает данные из базы данных. 4. Система фильтрует и структурирует информацию для формирования отчёта. 5. Администратор подтверждает параметры. 6. Система выводит предварительный вид отчёта. 7. Система создаёт файл отчёта и предлагает его для скачивания.
Альтернативные направления	1.1. Изменить параметры отчёта 1. Администратор решает изменить один или несколько параметров (например, группу или полугодие). 2. Система перезапрашивает параметры у пользователя. 3. Возврат к шагу 1 нормального поведения. 1.2. Просмотреть отчёт без экспорта 1. Администратор просматривает сформированный отчёт непосредственно в интерфейсе системы. 2. Отчёт остаётся в памяти системы. 3. Администратор может продолжить работу или завершить кейс.
Исключения	1.0.E1 Ошибка при формировании отчёта 1. Система сталкивается с внутренней ошибкой при генерации данных. 2. Система выводит сообщение: "Ошибка формирования отчёта. Попробуйте снова". 3. Администратор может повторить попытку или обратиться в техподдержку.
Приоритет	Средний
Частота использования	Около 20 обращений в день. Наибольшая нагрузка - в конце семестра перед аттестацией студентов.
Бизнес-правила	BR-9, BR-14, BR-17, BR-21

Идентификатор и название варианта использования	UC-4. Сформировать отчет.
Другая информация	1. Администратор должен иметь возможность сохранить параметры последнего запроса для повторного использования. 2. Система должна предоставлять возможность распечатать отчёт без сохранения файла (в будущих версиях). 3. Отчёт должен содержать графическое представление данных (диаграммы, средние баллы и т. д.).
Предположения	Предполагается, что 25% администраторов будут использовать данную функцию регулярно. Частота использования возрастает в 2 раза в аттестационный период. (Источник: тестирование системы в течение 2 месяцев)

6. Технические требования

6.1. Требования к программному обеспечению

Требования к программному обеспечению определяют, какие технологии и компоненты должны использоваться для реализации модуля, чтобы обеспечить его надёжность, производительность, безопасность и удобство использования.

- Использование C# и платформы .NET обеспечивает совместимость с операционной системой Windows и широкие возможности для работы с формами, базами данных и отчётами.
- Применение PostgreSQL гарантирует производительность и масштабируемость хранилища данных. Позволяет создать реляционную базу данных для хранения информации о студентах, группах, предметах и оценках.
- Разделение логики на формы и сервисы облегчает сопровождение и тестирование системы.
- IDE Visual Studio или Visual Studio Code позволяет удобно вести разработку и тестирование.

Таблица 3.

№ п/п	Наименование требования	Содержание требования
1	Назначение программы	Автоматизация учёта успеваемости студентов, включая выставление оценок, просмотр данных и расчёт среднего балла.
2	Язык программирования	C# (C Sharp)
3	Технология разработки	Windows Forms (WinForms)
4	СУБД	PostgreSQL
5	Совместимость ОС	Microsoft Windows 10 и выше
6	Кроссплатформенность	Не требуется (ориентировано на OS Windows).
7	Надёжность	Программа должна обрабатывать возможные ошибки (некорректный ввод, отсутствие соединения с БД) без аварийного завершения.
8	Безопасность	Ограничение доступа к данным путём разграничения прав пользователей и

		защиты подключения к БД (с использованием авторизации).
9	Производительность	Обработка запросов к БД и генерация отчётов должны выполняться без ощутимых задержек (до 3 секунд при обычной нагрузке).
10	Расширяемость	Возможность добавления новых форм, отчётов и модулей без существенной переработки архитектуры.
11	Масштабируемость	Возможность работы с увеличивающимся объёмом данных без потери производительности (до 10 000 записей и более).
12	Способ установки	Локальная установка с помощью установщика или ручное развертывание (копирование файлов и настройка строки подключения к БД).
13	Язык интерфейса	Русский

6.2. Требования к организационному обеспечению

Организационное обеспечение определяет условия внедрения, эксплуатации, администрирования и сопровождения программного продукта, а также обучение персонала для эффективной работы с системой учёта успеваемости студентов.

- Пользователи имеют дифференцированный доступ: администратор обладает полным контролем над системой, преподаватель может выставять оценки и просматривать данные своей группы, а студент имеет возможность просмотра своих оценок.
- Регулярное автоматическое резервное копирование базы данных (ежедневно) снижает риск потери информации при сбоях.
- Перед началом эксплуатации системы проводится инструктаж сотрудников по работе с модулем, включая описание интерфейса и основных функций.
- В системе ведётся логирование операций (выставление оценок, изменение данных) для контроля корректности работы.
- Система доступна круглосуточно, за исключением времени технического обслуживания (по графику).

Таблица 4.

№	Наименование требования	Содержание требования
1	Ответственные исполнители	Назначаются администратор системы и преподаватель, ответственный за сопровождение ПО.
2	Права доступа пользователей	Дифференцированный доступ: администратор (полный), преподаватель (частичный), студент (только просмотр).
3	Условия эксплуатации	Рабочее место пользователя - ПК с Windows 10 и выше, установленным .NET Framework.
4	Регламент обслуживания	Обслуживание выполняется системным администратором 1 раз в полгода или при сбоях.
5	Обучение пользователей	Перед вводом в эксплуатацию проводится инструктаж сотрудников по работе с модулем.
6	Защита и резервное копирование данных	Ежедневное автоматическое резервное копирование базы данных.
7	Документационное обеспечение	Включает руководство пользователя, техническую документацию, журнал регистрации ошибок.
8	Аудит действий пользователей	В системе ведётся логирование операций (выставление оценок, изменения данных).
9	Регламент обновлений	Обновление версии ПО проводится не чаще 1 раза в месяц по согласованию с администрацией.
10	Режим доступа к системе	Рабочее время системы - круглосуточно, кроме времени технического обслуживания (по графику).

7. Оценка трудоемкости разработки ПО на основе вариантов использования

7.1. Определение весовых показателей действующих лиц

1. Преподаватель — сложное действующее лицо.
2. Администратор — сложное действующее лицо.
3. Студент — простое действующее лицо.

Весовые коэффициенты действующих лиц

Тип действующего лица	Весовой коэффициент k_{ai}
простое	1
среднее	2
сложное	3

Рис. 8. Весовые коэффициенты действующих лиц

Таким образом, все три типа пользователей (Преподаватель, Администратор и Студент) имеют следующие весовые коэффициенты:

Преподаватель: $k_{ai} = 3$

Администратор: $k_{ai} = 3$

Студент: $k_{ai} = 1$

Общий весовой коэффициент равен:

$$A = 1 \times 1 + 3 \times 2 = 7$$

7.2. Определение весовых показателей вариантов использования

$$UC = \sum m_i \times k_{ai}$$

Варианты использования и их транзакции

Преподаватель:

1. Выставить оценку (5 транзакций) — Сложный
2. Просмотреть оценки (4 транзакции) — Средний
3. Просмотреть средний балл (3 транзакции) — Простой

Администратор:

1. Сформировать отчет (6 транзакций) — Сложный

7.3. Подсчет количества вариантов использования каждого типа

Простые:

1. УС-4. Просмотреть средний балл (Преподаватель)

Количество простых вариантов = 1

Средние:

1. УС-3. Просмотреть оценки (Преподаватель)

Количество средних вариантов = 1

Сложные:

1. УС-2. Выставить оценку (Преподаватель)

2. УС-5. Сформировать отчёт (Администратор)

Количество сложных вариантов = 2

Применение весовых коэффициентов

Теперь применим весовые коэффициенты:

Простые: $1 \times 1 = 1$

Средние: $1 \times 2 = 2$

Сложные: $2 \times 3 = 6$

Расчёт трудоемкости:

$$УС = (1 \times 1) + (1 \times 2) + (2 \times 3) = 1 + 2 + 6 = 9$$

7.4. Определение технической сложности проекта

$$ТСФ = 0,6 + (0,01 \times (\sum T_i \times Вес_i))$$

Таблица 5.

Показатель	Описание	Вес	Значение
T1	Работа с базой данных	2	5
T2	Онлайн работа	1	3
T3	Производительность	1	4
T4	Наличие пользовательского интерфейса	1	5
T5	Безопасность	1	3
T6	Документирование	0,5	4
T7	Комплексные вычисления	1	3

$$ТСФ = 0,6 + (0,01 \times 37) = 0,6 + 0,37 = 0,97$$

7.5. Определение уровня квалификации разработчиков

Уровень квалификации разработчиков (EF – environmental factor) вычисляются с учетом следующих показателей F_i .

Таблица 6.

Показатель	Описание	Вес	Значение
F1	Опыт работы с БД	1	5
F2	Опыт разработки Windows-приложений	1	5
F3	Мотивация	1	5
F4	Частичная занятость	-1	0
F5	Использование C#	-1	5
F6	Устойчивость требований	2	5

$$EF = 1,4 + (-0,03 \times (\sum F_i \times Вес_i))$$

$$EF = 1,4 + (-0,03 \times 20) = 1,4 - 0,6 = 0,8$$

7.6. Оценка трудоемкости проекта

$$UCP = UUCP \times TCF \times EF$$

$$UCP = 9 \times 0,97 \times 0,8 = 6,984 \text{ человеко-месяцев}$$

Если принять, что 20 человеко-часов на 1 UCP, то:

$$\text{Общая трудоемкость} = 6,984 \times 20 = 139,68 \text{ чел.-ч}$$

Это составляет при 40-часовой рабочей неделе: $139,68 / 40 \approx 3,5$ месяцев

8. Информационное обеспечение системы

Информационное обеспечение включает описание структуры данных, объёма и форм представления информации, а также источники и приёмники данных, которые используются в системе учёта успеваемости студентов.

- Источники информации: Данные о студентах, группах, предметах и оценках вводятся преподавателями и администраторами. Администраторы также управляют доступом к системе и формируют отчёты.
- Приёмник информации: База данных PostgreSQL хранит все данные системы, включая информацию о студентах, группах, предметах, выставленных оценках и сформированных отчётах.
- Организация данных: Информация логически структурирована в виде таблиц с чёткими связями (см. физическую модель на рис. 9) и представлена в удобных форматах для анализа и генерации отчётов.

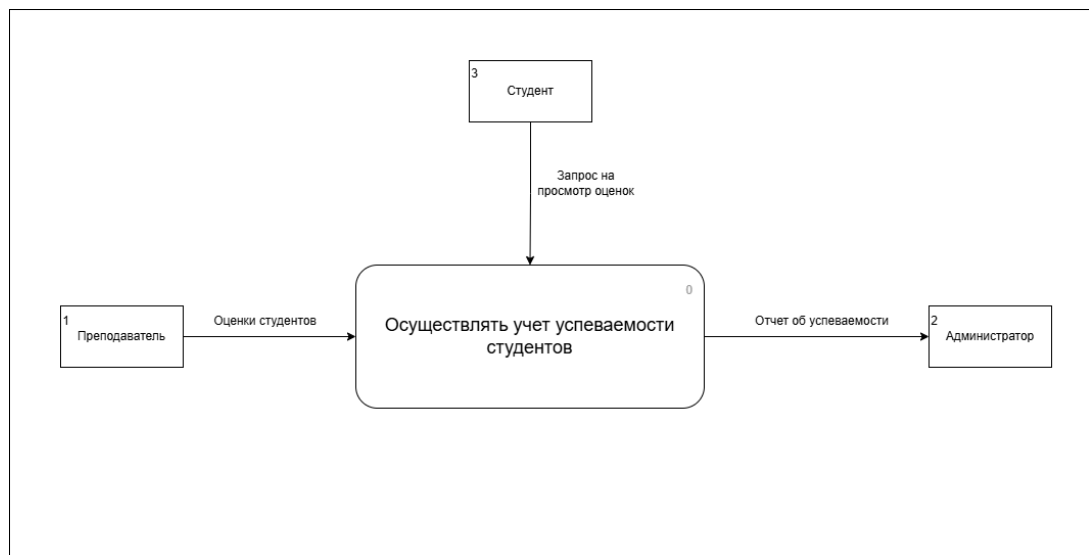


Рис. 9. Контекстная диаграмма потоков данных.

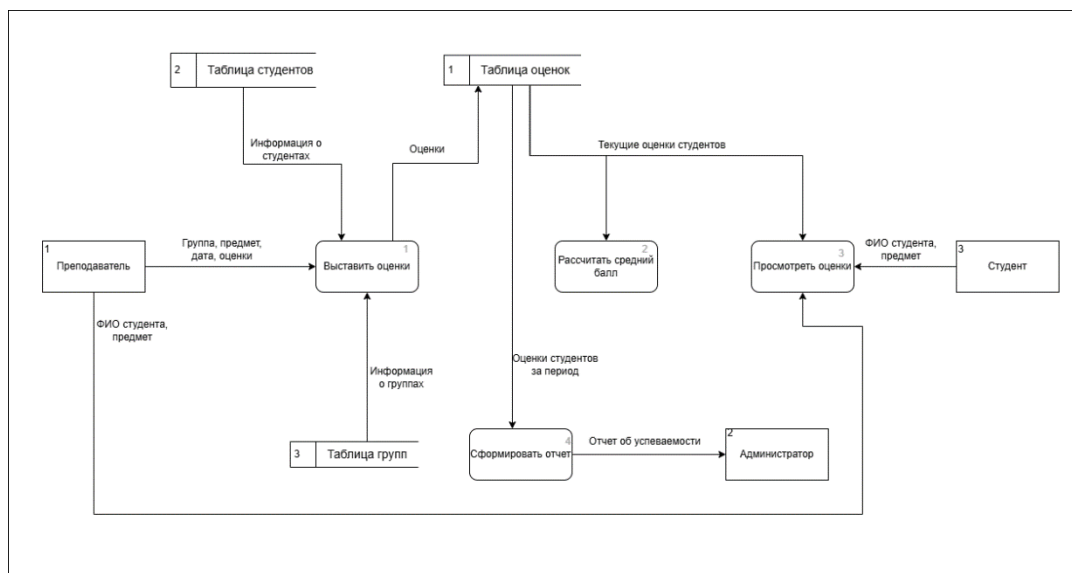


Рис. 10. Детализированная диаграмма потоков данных.

Таблица 7.

№	Наименование требования	Содержание требования
1	Состав и структура информации	В системе используются следующие виды данных: — данные о студентах (ФИО, группа, дата рождения); — данные о предметах и группах; — оценки студентов за учебный период; — отчёты об успеваемости.
2	Классификация и кодирование	Для уникальной идентификации используются: — Уникальные числовые идентификаторы для студентов, предметов и групп; — формат даты ГГГГ-ММ-ДД;
3	Формы представления информации	Информация отображается в виде: — таблиц (DataGridView); — форм ввода данных (WinForms);

4	Источники входной информации	Входная информация включает: — ФИО, группа, дата рождения студента; — название предмета; — дата выставления оценки;
5	Выходная информация	Выходная информация включает: — список студентов с оценками; — отчёты по успеваемости; — экспорт данных в PDF.
6	Назначение выходной информации	Используется для анализа успеваемости, составления отчётов и принятия решений.
7	Хранение информации	Данные хранятся в PostgreSQL в виде таблиц: — Students (студенты); — groups (группы); — grades (оценки). Хранение должно обеспечивать целостность и отказоустойчивость.
8	Объём хранимой информации	Объём данных рассчитывается исходя из числа студентов и предметов: ориентировочно до 10 000 записей в таблицах, объём базы - до 100 МБ.
9	Обновление и удаление данных	Операции выполняются через интерфейс администратора. Логика обеспечивает контроль целостности данных.
10	Архивирование информации	Возможность выгрузки данных в архив (PDF-файлы отчётов) и экспорта БД в формате SQL.
11	Надёжность обмена данными	При передаче данных применяются проверки на корректность ввода и обработки ошибок.

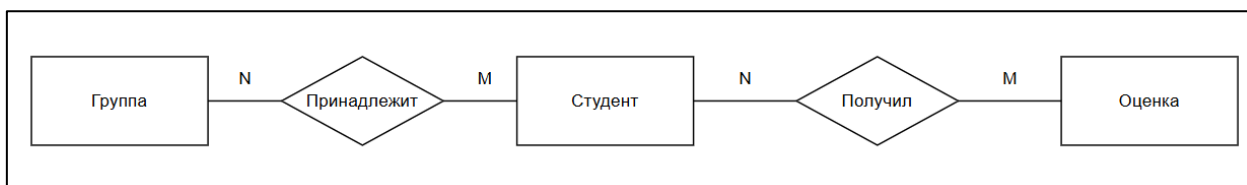


Рис 11. Диаграмма сущность связь

Таблица 8.

Сущность 1	Связь	Сущность 2	Тип связи
groups	1 ко многим	students	Одна группа может включать множество студентов
grades	1 ко многим	students	Один студент может получать множество оценок

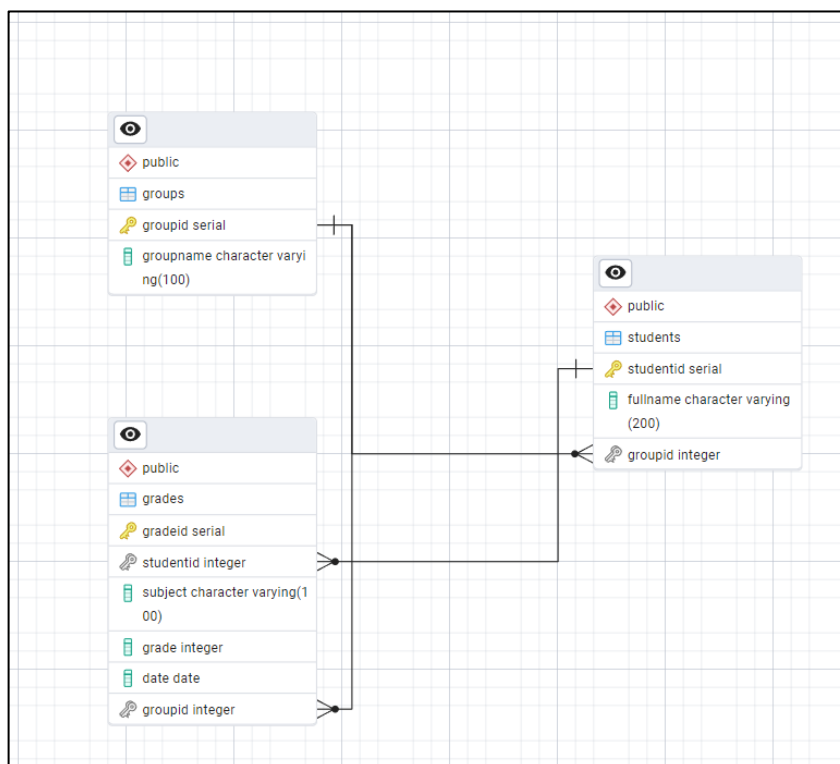


Рис. 12. Физическая модель базы данных.

Система должна использовать современные реляционные базы данных для хранения информации о студентах, группах, предметах и оценках. Для защиты целостности данных применяются встроенные механизмы СУБД. Структура базы данных оптимизирована для

исключения несанкционированного доступа к полной выгрузке данных. Таблицы (students, groups, grades) организованы по принципу нормализации, что уменьшает риск утечки конфиденциальной информации.

Описание сущности groups

Таблица 9.

№	Название атрибута	Тип данных	Значение	Допустимые значения	Описание
1	groupid	INTEGER	AUTO_INCREMENT	1-100 символов	Уникальный идентификатор группы
2	groupname	VARCHAR (100)	NOT_NULL	1-100 символов	Название учебной группы

Описание сущности students

Таблица 10.

№	Название атрибута	Тип данных	Значение	Допустимые значения	Описание
1	studentid	INTEGER	AUTO_INCREMENT	1-100 символов	Уникальный идентификатор студента
2	fullname	VARCHAR (100)	NOT_NULL	1-200 символов	ФИО студента
3	groupid	INTEGER	FOREIGN_KEY	Ссылка на groups.groupid	Идентификатор группы студента

Описание сущности grades

Таблица 11.

№	Название атрибута	Тип данных	Значение	Допустимые значения	Описание
1	gradeid	INTEGER	AUTO_INCREMENT	1-100 символов	Уникальный идентификатор оценки
2	studentid	INTEGER	FOREIGN_KEY	Ссылка на students.studentid	Идентификатор студента
3	subject	VARCHAR (100)	NOT_NULL	1-100 символов	Название предмета
4	grade	INTEGER	NULLABLE	2-5 (целые числа)	Оценка студента за предмет
5	date	DATE	NOT_NULL	Дата в формате ГГГГ-ММ-ДД	Дата выставления оценки
6	groupid	INTEGER	FOREIGN_KEY	Ссылка на groups.groupid	Идентификатор группы

9. Метрика спроектированной системы

9.1. Описание классов

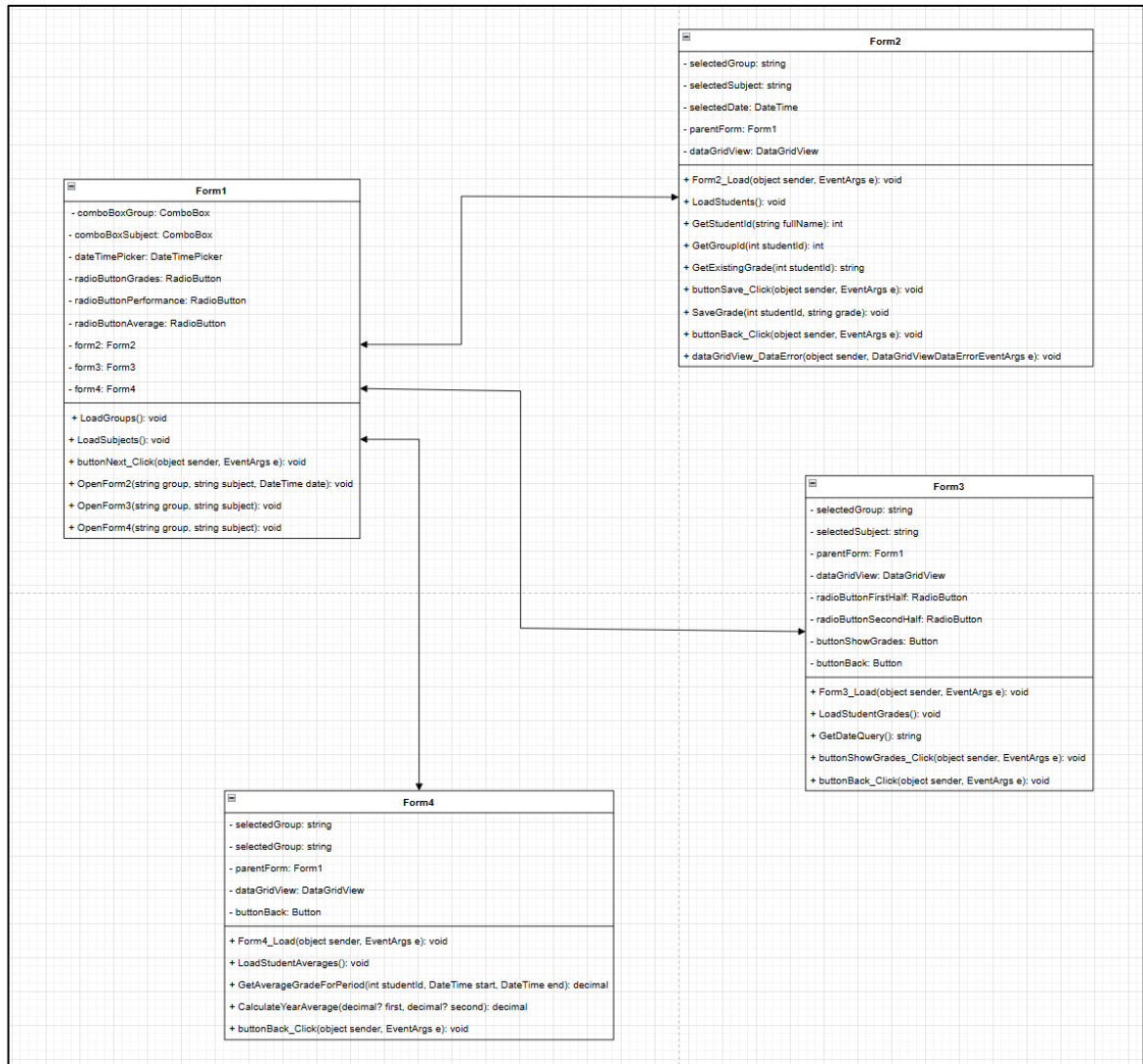


Рис 13. UML диаграмма классов

Класс Form1

Таблица 12.

Метод	Описание
LoadGroups ()	Загружает список групп из базы данных PostgreSQL
LoadSubjects ()	Загружает список предметов из базы данных PostgreSQL
buttonNext_Click (object sender, EventArgs e)	Обработчик события нажатия кнопок

Класс Form2

Таблица 13.

Метод	Описание
Form2_Load (object sender, EventArgs e)	Обработчик события загрузки формы. Инициализирует интерфейс и загружает данные студентов из базы данных
LoadStudents ()	Загружает список студентов выбранной группы из БД
GetStudentId (string fullName)	Возвращает идентификатор студента по его ФИО
GetGroupId(int studentId)	Возвращает идентификатор группы студента по его ID
GetExistingGrade (int studentId)	Получает текущую оценку студента из БД
buttonSave_Click (object sender, EventArgs e)	Обработчик нажатия кнопок

Класс Form3

Таблица 14.

Метод	Описание
Form3_Load (object sender, EventArgs e)	Обработчик события загрузки формы. Инициализирует интерфейс и загружает данные оценок
LoadStudentGrades ()	Загружает список оценок студентов из базы данных
GetDateQuery ()	Формирует SQL-запрос для получения оценок за выбранный период (первое или второе полугодие)
buttonShowGrades_Click (object sender, EventArgs e)	Обработчик нажатия кнопок

Таблица 15.

Метод	Описание
Form4_Load (object sender, EventArgs e)	Обработчик события загрузки формы. Инициализирует интерфейс и загружает данные студентов из базы данных
LoadStudentAverages ()	Загружает список студентов и их средние оценки за выбранный период
GetAverageGradeForPeriod (int studentId, DateTime start, DateTime end)	Получает среднюю оценку студента за указанный период времени
CalculateYearAverage (decimal? firstHalf, decimal? secondHalf)	Рассчитывает средний балл за год, учитывая отсутствие одного полугодия
buttonBack_Click (object sender, EventArgs e)	Обработчик нажатия кнопок

На основе диаграммы классов и структуры программного обеспечения проведем расчет метрик для ключевых классов системы. Результаты представлены в таблице и детально объяснены ниже.

Таблица 16.

Имя класса	Взвешенные методы на класс	Высота дерева наследования	Количество детей	Сцепление между классами	Отклик для класса	Недостаток связности
Form1	6	0	0	3	3	0
Form2	8	0	0	5	3	2
Form3	7	0	0	4	3	1
Form4	5	0	0	2	3	1

9.2. Расчёт метрики

Расчёт метрик для класса Form1.

Вложенные методы: 6 (LoadGroups, LoadSubjects, buttonNext_Click, OpenForm2, OpenForm3, OpenForm4).

Сцепление: 3 (методы OpenForm2, OpenForm3, OpenForm4 вызываются из Form1).

Отклик: $6 - 3 = 3$

LCOM:

$P = 6$

$Q = 3$ (пары методов с общими вызовами: OpenForm2–OpenForm3, OpenForm3–OpenForm4, OpenForm2–OpenForm4)

$LCOM = P - Q = 6 - 3 = 3$

Расчёт метрик для класса Form2.

Вложенные методы: 8 (LoadStudents, SaveGrade, buttonSave_Click, GetStudentId, GetExistingGrade, buttonBack_Click, Form2_Resize, ResizeDataGridView, dataGridView_DataError).

Сцепление: 5 (методы LoadStudents, SaveGrade, buttonBack_Click вызываются из Form1; GetStudentId, GetExistingGrade используются внутри класса).

Отклик: $8 - 5 = 3$

LCOM:

$P = 8$

$Q = 5$ (пары методов с общими вызовами: LoadStudents–SaveGrade, SaveGrade–buttonSave_Click, buttonBack_Click–Form2_Resize, Form2_Resize–ResizeDataGridView, ResizeDataGridView–dataGridView_DataError)

$LCOM = 8 - 5 = 3$

Расчёт метрик для класса Form3.

Вложенные методы: 7 (LoadStudentGrades, buttonShowGrades_Click, buttonBack_Click, Form3_Load, GetDataQuery, radioButtonFirstHalf_CheckedChanged, radioButtonSecondHalf_CheckedChanged).

Сцепление: 4 (методы LoadStudentGrades, buttonShowGrades_Click, buttonBack_Click вызываются из Form1; GetDataQuery используется внутри класса).

Отклик: $7 - 4 = 3$

LCOM:

$$P = 7$$

$Q = 4$ (пары методов с общими вызовами: LoadStudentGrades–buttonShowGrades_Click, buttonShowGrades_Click–buttonBack_Click, buttonBack_Click–Form3_Load, GetDateQuery–radioButtonFirstHalf_CheckedChanged)

$$LCOM = 7 - 4 = 3$$

Расчёт метрик для класса Form4.

Вложенные методы: 5 (LoadStudentAverages, buttonBack_Click, Form4_Load, GetAcademicYearDateRanges, CalculateYearAverage).

Сцепление: 2 (методы LoadStudentAverages, buttonBack_Click вызываются из Form1).

$$\text{Отклик: } 5 - 2 = 3$$

LCOM:

$$P = 5$$

$Q = 2$ (пары методов с общими вызовами: LoadStudentAverages–buttonBack_Click, Form4_Load–GetAcademicYearDateRanges)

$$LCOM = 5 - 2 = 3$$

10. Руководство пользователя

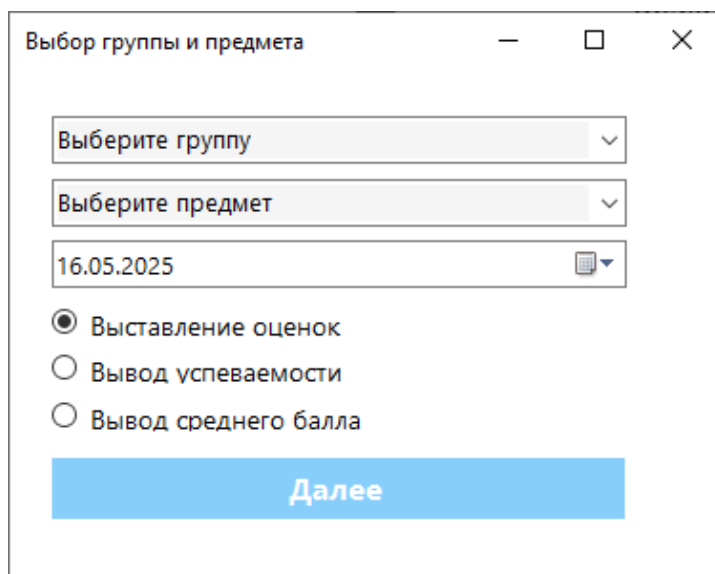
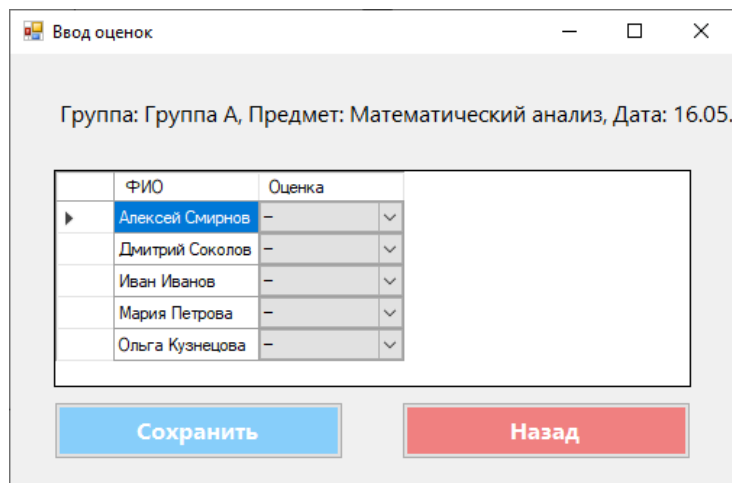


Рис 14. Форма 1: Выбор группы и предмета

На рисунке 12 изображена главная экранная форма учёта успеваемости студентов. Данная форма необходима для выбора необходимых параметров (группы, предмета и даты), а также позволяет пользователю выбрать дальнейшее действие (выставить оценку, вывести успеваемость, вывести средний балл). После выбора параметров, необходимых пользователю, открываются следующие экранные формы.



ФИО	Оценка
Алексей Смирнов	—
Дмитрий Соколов	—
Иван Иванов	—
Мария Петрова	—
Ольга Кузнецова	—

Рис 15. Форма 2: Выставление оценок

Следующая экранная форма – форма выставления оценок (рис. 13). Здесь выводится список студентов выбранной ранее группы, а также представляется возможность выставить оценки. Пользователь может сохранить внесенные оценки в базу данных или вернуться на главную форму.

Успеваемость студентов

Группа: Группа А, Предмет: Математический анализ

☒ 1 полугодие ☐ 2 полугодие

Показать

Назад

Рис 16. Форма 3: Просмотр успеваемости (вид 1)

Успеваемость студентов

Группа: Группа А, Предмет: Математический анализ

☐ 1 полугодие ☒ 2 полугодие

Показать

	ФИО студента	12.02.202	05.03.202	18.03.202	31.03.202	06.04.202	22.04.202
▶	Иван И...	–	–	3	5	–	–
	Мария ...	–	3	–	–	–	–
	Алексе...	4	–	–	–	–	–
	Ольга ...	–	–	–	–	3	5
	Дмитри...	–	–	–	–	–	3

Назад

Рис 17. Форма 3: Просмотр успеваемости (вид 2)

Далее представлены два вида третьей экранной формы – просмотр успеваемости студентов (рис 14 – 15). Изначально пользователь не видит таблицы с оценками, так как необходимо выбрать полугодие, за которое он желает просмотреть успеваемость. После

нажатия кнопки «Показать» выводится таблица с оценками студентов группы, выбранной в главной форме. Также пользователь может вернуться на главную форму.

	ФИО студента	Средний балл (1 полугодие)	Средний балл (2 полугодие)	Средний балл (за год)
▶	Анна Васильева	4,33	4,25	4,29
	Елена Федорова	4,00	3,50	3,75
	Максим Попов	4,00	5,00	4,50
	Петр Николаев	3,67	4,00	3,83
	Светлана Михай...	3,50	3,00	3,25

Назад

Рис 18. Форма 4: Просмотр средних баллов

Четвертая форма – просмотр среднего балла (рис. 16). Здесь пользователь может просмотреть средние баллы всех студентов выбранной группы за полугодия и год. Пользователь также может вернуться на главную форму.

11. Структура приложения

11.1. Архитектура приложения

Архитектура приложения основана на трёхуровневой (трёхслойной) модели, включающей:

- Уровень представления (UI) — реализован с использованием Windows Forms для взаимодействия с пользователем.
- Уровень бизнес-логики (BLL) — содержит сервисные классы, выполняющие обработку данных (например, Form2, Form3, Form4).
- Уровень доступа к данным (DAL) — реализован с использованием PostgreSQL и классов взаимодействия с БД (DB.cs, SQL-запросы через NpgsqlCommand).

Таблица 17.

№	Название подсистемы/ модуля	Назначение	Связи с другими модулями	Программные средства реализации
1	Модуль пользовательского интерфейса (UI)	Обеспечивает взаимодействие пользователя с системой через формы.	Взаимодействует с модулем бизнес-логики (BLL).	C#, Windows Forms
2	Модуль обработки оценок	Выполняет выставление оценок, проверку корректности данных и взаимодействие с БД.	Связан с UI для получения данных и с DAL для записи в БД.	C#, классы бизнес-логики (Form2, DB.cs)
3	Модуль анализа успеваемости	Рассчитывает средний балл за год, формирует отчёты по	Использует данные из DAL и передаёт результаты в UI.	C#, методы в Form4

		группам и студентам.		
4	Модуль доступа к данным (DAL)	Управляет подключением к PostgreSQL, выполняет CRUD-операции.	Используется всеми модулями для работы с БД.	PostgreSQL, Npgsql.Data, C#
5	Модуль валидации данных	Проверяет корректность ввода.	Интегрирован в BLL для предотвращения ошибок.	C#, методы в Form2, Form3

11.2. Листинг программы

Класс Form1

```

public partial class Form1 : Form
{
    private Form2 form2;
    private Form3 form3;
    private Form4 form4;

    public Form1()
    {
        InitializeComponent();
        LoadGroups();
        LoadSubjects();
        comboBoxGroup.Text = "Выберите группу";
        comboBoxSubject.Text = "Выберите предмет";

        buttonNext.Click += buttonNext_Click;
    }

    private void LoadGroups()
    {
        using (var conn = new
NpgsqlConnection("Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres"))
        {
            conn.Open();
            string query = "SELECT DISTINCT groupname FROM groups ORDER BY groupname";
            using (var cmd = new NpgsqlCommand(query, conn))
            using (var reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    comboBoxGroup.Items.Add(reader.GetString(0));
                }
            }
        }
    }

    private void LoadSubjects()

```

```

{
    comboBoxSubject.Items.Add("Математический анализ");
    comboBoxSubject.Items.Add("Программирование");
    comboBoxSubject.Items.Add("Схемотехника");
    comboBoxSubject.Items.Add("Базы данных");
    comboBoxSubject.Items.Add("Операционные системы");
}

private void buttonNext_Click(object sender, EventArgs e)
{
    string selectedGroup = comboBoxGroup.Text;
    string selectedSubject = comboBoxSubject.Text;
    DateTime selectedDate = dateTimePicker.Value;

    List<string> emptyFields = new List<string>();

    if (string.IsNullOrEmpty(selectedGroup) || selectedGroup == "Выберите группу")
    {
        emptyFields.Add("группу");
    }

    if (string.IsNullOrEmpty(selectedSubject) || selectedSubject == "Выберите предмет")
    {
        emptyFields.Add("предмет");
    }

    if (emptyFields.Count > 0)
    {
        string missingFields = string.Join(", ", emptyFields);
        MessageBox.Show($"Пожалуйста, выберите {missingFields}.");
        return;
    }

    if (radioButtonGrades.Checked)
    {
        OpenForm2(selectedGroup, selectedSubject, selectedDate);
    }
    else if (radioButtonPerformance.Checked)
    {
        OpenForm3(selectedGroup, selectedSubject);
    }
    else if (radioButtonAverage.Checked)
    {
        OpenForm4(selectedGroup, selectedSubject);
    }
    else
    {
        MessageBox.Show("Пожалуйста, выберите одно из действий.");
    }
}

private void OpenForm2(string selectedGroup, string selectedSubject, DateTime selectedDate)
{
    if (form2 == null || form2.IsDisposed)
    {
        form2 = new Form2(this, selectedGroup, selectedSubject, selectedDate);
        form2.FormClosed += (sender, e) => this.Show();
    }

    form2.Show();
    this.Hide();
}

private void OpenForm3(string selectedGroup, string selectedSubject)
{
    this.Hide();

    if (form3 == null || form3.IsDisposed)
    {

```

```

        form3 = new Form3(this, selectedGroup, selectedSubject);
        form3.FormClosed += (sender, e) => this.Show();
    }

    form3.Show();
}

private void OpenForm4(string selectedGroup, string selectedSubject)
{
    this.Hide();

    if (form4 == null || form4.IsDisposed)
    {
        form4 = new Form4(this, selectedGroup, selectedSubject);
        form4.FormClosed += (sender, e) => this.Show();
    }

    form4.Show();
}
}

```

Класс Form2

```

public partial class Form2 : Form
{
    private readonly string selectedGroup;
    private readonly string selectedSubject;
    private readonly DateTime selectedDate;
    private readonly Form1 parentForm;

    public bool SavedSuccessfully { get; private set; }

    public Form2(Form1 parent, string group, string subject, DateTime date)
    {
        InitializeComponent();
        selectedGroup = group;
        selectedSubject = subject;
        selectedDate = date;
        parentForm = parent;

        Load += Form2_Load;
        Resize += Form2_Resize;
        buttonSave.Click += ButtonSave_Click;
        buttonBack.Click += buttonBack_Click;
    }

    private void Form2_Load(object sender, EventArgs e)
    {
        labelGroupSubject.Text = $"Группа: {selectedGroup}, Предмет: {selectedSubject}, Дата: {selectedDate:dd.MM.yyyy}";

        dataGridView.Rows.Clear();
        dataGridView.Columns.Clear();

        dataGridView.Columns.Add("ФИО", "ФИО");

        var gradeColumn = new DataGridViewComboBoxColumn
        {
            Name = "Оценка",
            HeaderText = "Оценка",
            DataSource = new string[] { "—", "2", "3", "4", "5" }
        };
        dataGridView.Columns.Add(gradeColumn);

        LoadStudents();

        ResizeDataGridView();
    }
}

```

```

private void LoadStudents()
{
    string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

    using (var connection = new NpgsqlConnection(connectionString))
    {
        connection.Open();

        string query = @"
            SELECT s.studentid, s.fullname
            FROM students s
            JOIN groups g ON s.groupid = g.groupid
            WHERE g.groupname = @groupname
            ORDER BY s.fullname";

        using (var cmd = new NpgsqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("groupname", selectedGroup);

            using (var reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    int studentId = reader.GetInt32(0);
                    string fullName = reader.GetString(1);
                    string grade = GetExistingGrade(studentId);

                    dataGridView.Rows.Add(fullName, grade ?? "—");
                }
            }
        }
    }
}

private string GetExistingGrade(int studentId)
{
    string grade = null;
    string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

    using (var connection = new NpgsqlConnection(connectionString))
    {
        connection.Open();
        string query = @"
            SELECT grade
            FROM grades
            WHERE studentid = @studentid
            AND subject = @subject
            AND date = @date";

        using (var cmd = new NpgsqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("studentid", studentId);
            cmd.Parameters.AddWithValue("subject", selectedSubject);
            cmd.Parameters.AddWithValue("date", selectedDate);

            object result = cmd.ExecuteScalar();
            grade = result != null ? result.ToString() : null;
        }
    }

    return grade;
}

private void ButtonSave_Click(object sender, EventArgs e)
{
    bool anyChangesMade = false; //1

    for (int i = 0; i < dataGridView.Rows.Count; i++) //2
    {

```

```

string fullName = dataGridView.Rows[i].Cells[0].Value?.ToString();//3
string grade = dataGridView.Rows[i].Cells[1].Value?.ToString();//4

if (!string.IsNullOrEmpty(fullName) && grade != "—")//5
{
    int studentId = GetStudentId(fullName);//6
    string existingGrade = GetExistingGrade(studentId);//7

    if (existingGrade != grade)//8
    {
        SaveGrade(studentId, grade);//9
        anyChangesMade = true;
    }
}

if (anyChangesMade)//10
{
    SavedSuccessfully = true;

    MessageBox.Show("Оценки сохранены.", "Сохранение", MessageBoxButtons.OK, MessageBoxIcon.Information);

    this.Close();
    parentForm.Show();
    return;
}

MessageBox.Show("Нет изменений для сохранения.", "Сохранение", MessageBoxButtons.OK,
MessageBoxIcon.Information);//11
}

private void SaveGrade(int studentId, string grade)
{
    int? gradeValue = null;

    if (grade != "—")
    {
        gradeValue = Convert.ToInt32(grade);
    }

    string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

    using (var connection = new NpgsqlConnection(connectionString))
    {
        connection.Open();

        string checkQuery = @"
            SELECT COUNT(*)
            FROM grades
            WHERE studentid = @studentid
            AND subject = @subject
            AND date = @date";

        using (var cmdCheck = new NpgsqlCommand(checkQuery, connection))
        {
            cmdCheck.Parameters.AddWithValue("studentid", studentId);
            cmdCheck.Parameters.AddWithValue("subject", selectedSubject);
            cmdCheck.Parameters.AddWithValue("date", selectedDate);

            int count = Convert.ToInt32(cmdCheck.ExecuteScalar());

            if (count > 0)
            {
                string updateQuery = @"
                    UPDATE grades
                    SET grade = @grade
                    WHERE studentid = @studentid
                    AND subject = @subject
                    AND date = @date";
            }
        }
    }
}

```

```

        using (var cmdUpdate = new NpgsqlCommand(updateQuery, connection))
        {
            cmdUpdate.Parameters.AddWithValue("grade", gradeValue.HasValue ? gradeValue.Value :
(object)DBNull.Value);
            cmdUpdate.Parameters.AddWithValue("studentid", studentId);
            cmdUpdate.Parameters.AddWithValue("subject", selectedSubject);
            cmdUpdate.Parameters.AddWithValue("date", selectedDate);

            cmdUpdate.ExecuteNonQuery();
        }
    }
    else
    {
        string insertQuery = @"
            INSERT INTO grades (studentid, subject, grade, date, groupid)
            VALUES (@studentid, @subject, @grade, @date, @groupid);

        using (var cmdInsert = new NpgsqlCommand(insertQuery, connection))
        {
            cmdInsert.Parameters.AddWithValue("studentid", studentId);
            cmdInsert.Parameters.AddWithValue("subject", selectedSubject);
            cmdInsert.Parameters.AddWithValue("grade", gradeValue.HasValue ? gradeValue.Value :
(object)DBNull.Value);
            cmdInsert.Parameters.AddWithValue("date", selectedDate);
            cmdInsert.Parameters.AddWithValue("groupid", GetGroupId(studentId));

            cmdInsert.ExecuteNonQuery();
        }
    }
}

private int GetStudentId(string fullName)
{
    int studentId = -1;
    string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

    using (var connection = new NpgsqlConnection(connectionString))
    {
        connection.Open();
        string query = "SELECT studentid FROM students WHERE fullname = @fullname";

        using (var cmd = new NpgsqlCommand(query, connection))
        {
            cmd.Parameters.AddWithValue("fullname", fullName);
            object result = cmd.ExecuteScalar();

            if (result != null)
                studentId = Convert.ToInt32(result);
        }
    }

    return studentId;
}

private int GetGroupId(int studentId)
{
    int groupId = -1;
    string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

    using (var connection = new NpgsqlConnection(connectionString))
    {
        connection.Open();
        string query = "SELECT groupid FROM students WHERE studentid = @studentid";

        using (var cmd = new NpgsqlCommand(query, connection))
        {

```

```

        cmd.Parameters.AddWithValue("studentid", studentId);
        object result = cmd.ExecuteScalar();

        if (result != null)
            groupId = Convert.ToInt32(result);
    }
}

return groupId;
}

private void buttonBack_Click(object sender, EventArgs e)
{
    this.Close();
    parentForm.Show();
}

private void Form2_Resize(object sender, EventArgs e)
{
    ResizeDataGridView();
}

private void ResizeDataGridView()
{
    if (dataGridView != null)
    {
        int tableWidth = ClientSize.Width - 60;
        dataGridView.Size = new System.Drawing.Size(tableWidth, 150);
        dataGridView.Location = new System.Drawing.Point(30, 80);
    }
}

private void dataGridView_DataError(object sender, DataGridViewDataErrorEventArgs e)
{
    MessageBox.Show("Пронизошла ошибка при вводе данных.");
    e.ThrowException = false;
}
}

```

Класс Form3

```

public partial class Form3 : Form
{
    private string selectedGroup;
    private string selectedSubject;
    private Form1 parentForm;

    public Form3(Form1 parent, string group, string subject)
    {
        InitializeComponent();
        selectedGroup = group;
        selectedSubject = subject;
        parentForm = parent;
    }

    private void Form3_Load(object sender, EventArgs e)
    {
        labelGroupSubject.Text = $"Группа: {selectedGroup}, Предмет: {selectedSubject}";
    }

    private void LoadStudentGrades()
    {
        dataGridView.Rows.Clear();
        dataGridView.Columns.Clear();

        string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

        using (var connection = new NpgsqlConnection(connectionString))

```



```

{
    connection.Open();

    string dateQuery = GetDateQuery();
    List<DateTime> uniqueDates = new List<DateTime>();
    using (var dateCommand = new NpgsqlCommand(dateQuery, connection))
    using (var reader = dateCommand.ExecuteReader())
    {
        while (reader.Read())
        {
            uniqueDates.Add(reader.GetDateTime(0));
        }
    }

    if (uniqueDates.Count == 0)
    {
        MessageBox.Show("За выбранный период нет оценок.");
        return;
    }

    dataGridView.Columns.Add("StudentName", "ФИО студента");
    foreach (var date in uniqueDates)
    {
        dataGridView.Columns.Add(date.ToShortDateString(), date.ToShortDateString());
    }

    string studentQuery = "SELECT fullname FROM students WHERE groupid = (SELECT groupid FROM groups
WHERE groupname = @groupname)";
    List<string> students = new List<string>();

    using (var studentCommand = new NpgsqlCommand(studentQuery, connection))
    {
        studentCommand.Parameters.AddWithValue("groupname", selectedGroup);
        using (var reader = studentCommand.ExecuteReader())
        {
            while (reader.Read())
            {
                students.Add(reader.GetString(0));
            }
        }
    }

    if (students.Count == 0)
    {
        MessageBox.Show($"Студенты для группы '{selectedGroup}' не найдены.");
        return;
    }

    foreach (var student in students)
    {
        var row = new DataGridViewRow();
        row.Cells.Add(new DataGridViewTextBoxCell { Value = student });

        foreach (var date in uniqueDates)
        {
            string gradeQuery = "SELECT grade FROM grades WHERE studentid = (SELECT studentid FROM students
WHERE fullname = @fullname) AND date = @date AND subject = @subject";
            using (var gradeCommand = new NpgsqlCommand(gradeQuery, connection))
            {
                gradeCommand.Parameters.AddWithValue("fullname", student);
                gradeCommand.Parameters.AddWithValue("date", date);
                gradeCommand.Parameters.AddWithValue("subject", selectedSubject);

                var result = gradeCommand.ExecuteScalar();
                row.Cells.Add(new DataGridViewTextBoxCell { Value = result?.ToString() ?? "—" });
            }
        }

        dataGridView.Rows.Add(row);
    }
}

```

```

    }
}
}

private string GetDateQuery()
{
    DateTime startDate;
    DateTime endDate;

    if (radioButtonFirstHalf.Checked)
    {
        startDate = new DateTime(DateTime.Now.Year, 9, 1);
        endDate = new DateTime(DateTime.Now.Year, 12, 31);
    }
    else
    {
        startDate = new DateTime(DateTime.Now.Year, 1, 1);
        endDate = new DateTime(DateTime.Now.Year, 5, 31);
    }

    return $"SELECT DISTINCT date FROM grades WHERE date BETWEEN '{startDate:yyyy-MM-dd}' AND '{endDate:yyyy-MM-dd}' AND subject = '{selectedSubject}' ORDER BY date";
}

private void buttonBack_Click(object sender, EventArgs e)
{
    this.Close();
    parentForm.Show();
}

private void buttonShowGrades_Click(object sender, EventArgs e)
{
    LoadStudentGrades();
}
}

```

Класс Form4

```

public partial class Form4 : Form
{
    private readonly string selectedGroup;
    private readonly string selectedSubject;
    private readonly Form1 parentForm;

    public Form4(Form1 parent, string group, string subject)
    {
        InitializeComponent();
        selectedGroup = group;
        selectedSubject = subject;
        parentForm = parent;

        Load += Form4_Load;
        buttonBack.Click += buttonBack_Click;
    }

    private void Form4_Load(object sender, EventArgs e)
    {
        labelGroupSubject.Text = $"Группа: {selectedGroup}, Предмет: {selectedSubject}";
        LoadStudentAverages();
    }

    private void LoadStudentAverages()
    {
        dataGridView.Rows.Clear();
        dataGridView.Columns.Clear();

        dataGridView.Columns.Add("StudentName", "ФИО студента");
        dataGridView.Columns.Add("FirstHalfAverage", "Средний балл (1 полугодие)");
    }
}

```

```

dataGridView.Columns.Add("SecondHalfAverage", "Средний балл (2 полугодие)");
dataGridView.Columns.Add("YearAverage", "Средний балл (за год)");

var dateRanges = GetAcademicYearDateRanges();

string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

using (var connection = new NpgsqlConnection(connectionString))
{
    connection.Open();

    string studentQuery = @"
        SELECT s.studentid, s.fullname
        FROM students s
        JOIN groups g ON s.groupid = g.groupid
        WHERE g.groupname = @groupname
        ORDER BY s.fullname";

    using (var cmdStudents = new NpgsqlCommand(studentQuery, connection))
    {
        cmdStudents.Parameters.AddWithValue("groupname", selectedGroup);

        using (var reader = cmdStudents.ExecuteReader())
        {
            while (reader.Read())
            {
                int studentId = reader.GetInt32(0);
                string fullName = reader.GetString(1);

                decimal? firstHalfAverage = GetAverageGradeForPeriod(studentId, dateRanges.firstHalfStart,
dateRanges.firstHalfEnd);
                decimal? secondHalfAverage = GetAverageGradeForPeriod(studentId, dateRanges.secondHalfStart,
dateRanges.secondHalfEnd);
                decimal? yearAverage = CalculateYearAverage(firstHalfAverage, secondHalfAverage);

                dataGridView.Rows.Add(
                    fullName,
                    firstHalfAverage?.ToString("F2") ?? "—",
                    secondHalfAverage?.ToString("F2") ?? "—",
                    yearAverage?.ToString("F2") ?? "—"
                );
            }
        }
    }
}

private (DateTime firstHalfStart, DateTime firstHalfEnd, DateTime secondHalfStart, DateTime secondHalfEnd)
GetAcademicYearDateRanges()
{
    int currentYear = DateTime.Now.Year;

    // Определяем начало учебного года
    int academicYearStart = DateTime.Now.Month >= 9 ? currentYear : currentYear - 1;

    DateTime firstHalfStart = new DateTime(academicYearStart, 9, 1);
    DateTime firstHalfEnd = new DateTime(academicYearStart, 12, 31);
    DateTime secondHalfStart = new DateTime(academicYearStart + 1, 1, 1);
    DateTime secondHalfEnd = new DateTime(academicYearStart + 1, 5, 31);

    return (firstHalfStart, firstHalfEnd, secondHalfStart, secondHalfEnd);
}

private decimal? GetAverageGradeForPeriod(int studentId, DateTime startDate, DateTime endDate)
{
    string connectionString = "Host=localhost;Port=5432;Username=postgres;Password=112233;Database=postgres";

    using (var connection = new NpgsqlConnection(connectionString))
    {

```

```

connection.Open();

string query = @"
    SELECT AVG(grade)
    FROM grades
    WHERE studentid = @studentid
    AND subject = @subject
    AND date BETWEEN @startdate AND @enddate";

using (var cmd = new NpgsqlCommand(query, connection))
{
    cmd.Parameters.AddWithValue("studentid", studentId);
    cmd.Parameters.AddWithValue("subject", selectedSubject);
    cmd.Parameters.AddWithValue("startdate", startDate);
    cmd.Parameters.AddWithValue("enddate", endDate);

    object result = cmd.ExecuteScalar();
    return result != null && result != DBNull.Value ? Convert.ToDecimal(result) : (decimal?)null;
}
}

private decimal? CalculateYearAverage(decimal? firstHalf, decimal? secondHalf)
{
    if (firstHalf.HasValue || secondHalf.HasValue)
    {
        List<decimal> averages = new List<decimal>();
        if (firstHalf.HasValue) averages.Add(firstHalf.Value);
        if (secondHalf.HasValue) averages.Add(secondHalf.Value);
        return averages.Average();
    }

    return null;
}

private void buttonBack_Click(object sender, EventArgs e)
{
    this.Close();
    parentForm.Show();
}
}

```

12. Тестирование

Для обеспечения надежности и корректности работы модуля учёта успеваемости студентов были проведены следующие виды тестирования:

- Структурное тестирование (метод ветвей и границ)
- Функциональное тестирование (метод эквивалентного разбиения)

Эти методы позволили оценить соответствие системы требованиям, выявить дефекты и убедиться в корректности работы ключевых функций.

12.1. Структурное тестирование

Для разработки тест-кейсов использовалась методика тест-дизайна ветвей и границ. Метод выполняет следующие действия:

1. Перебирает строки таблицы студентов.
2. Получает имя студента и оценку.
3. Сравнивает новую оценку со старой из базы данных.
4. При наличии отличий - сохраняет данные в БД.
5. После сохранения всех изменений - закрывает форму и возвращает пользователя к Form1.

Объект тестирования

Объектом тестирования является функция сохранения оценок, реализованная в методе `buttonSave_Click ()`, который вызывается при нажатии пользователем кнопки «Сохранить» на форме Form2.

```
private void buttonSave_Click(object sender, EventArgs e)
{
    bool anyChangesMade = false; //1

    for (int i = 0; i < dataGridView.Rows.Count; i++) //2
    {
        string fullName = dataGridView.Rows[i].Cells[0].Value?.ToString(); //3
        string grade = dataGridView.Rows[i].Cells[1].Value?.ToString(); //4

        if (!string.IsNullOrEmpty(fullName) && grade != "-") //5
        {
            int studentId = GetStudentId(fullName); //6
            string existingGrade = GetExistingGrade(studentId); //7

            if (existingGrade != grade) //8
            {
                SaveGrade(studentId, grade); //9
                anyChangesMade = true;
            }
        }
    }

    if (anyChangesMade) //10
    {
        SavedSuccessfully = true;

        MessageBox.Show("Оценки сохранены.", "Сохранение", MessageBoxButtons.OK, MessageBoxIcon.Information);

        this.Close();
        parentForm.Show();
        return;
    }

    MessageBox.Show("Нет изменений для сохранения.", "Сохранение", MessageBoxButtons.OK, MessageBoxIcon.Information); //11
}
```

Рис 19. Код тестируемого метода `buttonSave_Click ()`.

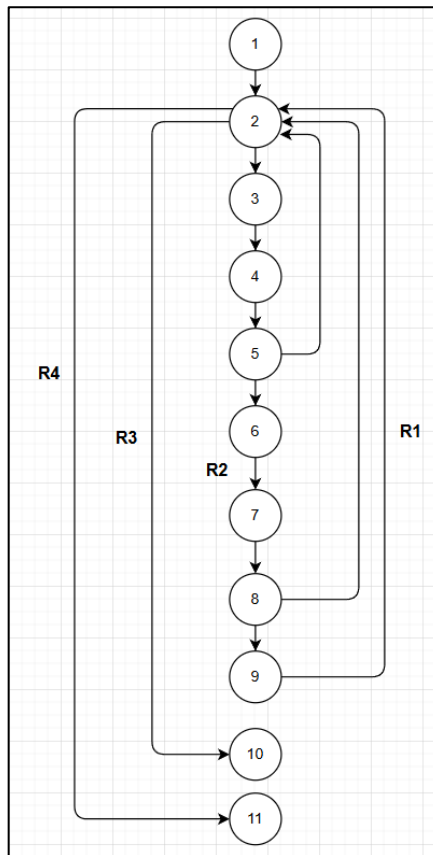


Рис 20. Потокный граф метода buttonSave_Click ().

Граф показывает все узлы и переходы между ними. Он используется для расчёта количества необходимых тест-кейсов.

Оценка сложности по МакКейбу

Для определения минимального количества независимых линейных путей, необходимого для полного структурного покрытия кода, используется оценка цикломатической сложности по формуле МакКейба:

$$V(G) = E - N + 2P$$

Где:

E - количество рёбер в потоковом графе

N - количество узлов

P - количество точек выхода из программы

Подставляем значения:

E = 13 - количество рёбер на потоковом графе

N = 11 - количество узлов

P = 1 - точка выхода из метода

Расчёт:

$$V(G) = 13 - 11 + 2 \times 1 = 4$$

На основе значения цикломатической сложности $V(G) = 4$ были определены следующие независимые линейные пути, соответствующие всем возможным условиям в коде:

Таблица 18.

№	Путь	Описание
P1	1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 2 - 10	Сохранение допустимой оценки, которая отличается от текущей
P2	1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 2 - 10	Оценка совпадает с уже существующей - сохранения нет
P3	1 - 2 - 3 - 4 - 5 - 2 - 10	Оценка не выбрана или равна «_» - строка пропущена
P4	1 - 2 - 11	Таблица студентов пустая - изменения не производились

Тест-кейсы

Путь P1: Сохранение допустимой оценки (отличающейся от текущей).

Таблица 19.

ID	Описание	Предусловия	Шаги
ТС_01	Сохранение минимальной оценки «2»	ФормаForm2 открыта, студент имеет оценку «3»	1. Открыть форму 2. Выбрать студента 3. Ввести оценку «2» 4. Нажать «Сохранить»

Таблица 20.

Ожидаемый результат	Код
Оценка изменена на «2», флаг anyChangesMade = true, форма закрыта	<pre> public void TC_01_SaveValidGrade_MinimumValue() { // Подготовка var form = new Form2(); var studentId = GetStudentId("Иванов"); // Студент с текущей оценкой "3" var existingGrade = GetExistingGrade(studentId); // Выполнение </pre>

	<pre> form.SaveGrade(studentId, "2"); // Проверка Assert.IsTrue(form.anyChangesMade, "Флаг изменений должен быть true"); var newGrade = GetExistingGrade(studentId); Assert.AreEqual("2", newGrade, "Оценка не сохранена корректно"); } </pre>
--	---

Путь P2: Пропуск одинаковых оценок.

Таблица 21.

ID	Описание	Предусловия	Шаги
ТС_02	Оценка совпадает с текущей	Форма Form2 открыта, студент имеет оценку «4»	1. Открыть форму 2. Выбрать студента 3. Оставить оценку «4» 4. Нажать «Сохранить»

Таблица 22.

Ожидаемый результат	Код
Сообщение: «Нет изменений для сохранения», форма остаётся открытой	<pre> public void TC_02_SkipIdenticalGrades() { // Подготовка var form = new Form2(); var studentId = GetStudentId("Соколов"); // Студент с текущей оценкой "4" var existingGrade = GetExistingGrade(studentId); // Выполнение form.SaveGrade(studentId, existingGrade); // Проверка Assert.IsFalse(form.anyChangesMade, "Флаг изменений должен быть false"); var result = GetExistingGrade(studentId); Assert.AreEqual(existingGrade, result, "Оценка должна остаться без изменений"); } </pre>

Путь Р3: Пропуск строки с пустой оценкой

Таблица 23.

ID	Описание	Предусловия	Шаги
TC_03	Пустая оценка в одной строке	Форма Form2 открыта, студент имеет оценку «—»	1. Открыть форму 2. Выбрать студента 3. Не выбирать оценку 4. Нажать «Сохранить»

Таблица 24.

Ожидаемый результат	Код
Строка пропущена, флаг anyChangesMade = false, вывод сообщения	<pre> public void TC_03_SkipEmptyGrade() { // Подготовка var form = new Form2(); var studentId = GetStudentId("Кузнецов"); // Студент с пустой оценкой ("—") // Выполнение form.buttonSave_Click(null, EventArgs.Empty); // Проверка Assert.IsFalse(form.anyChangesMade, "Флаг изменений должен быть false"); var grade = GetExistingGrade(studentId); Assert.AreEqual("-", grade, "Оценка должна остаться пустой"); } </pre>

Путь Р4: Таблица пустая

Таблица 25.

ID	Описание	Предусловия	Шаги	Ожидаемый результат
TC_04	Попытка сохранения при пустой таблице	Форма Form2 открыта, dataGridView.Rows.Count == 0	1. Открыть форму 2. Нажать «Сохранить»	Сообщение: «Нет изменений для сохранения», форма остаётся открытой

Таблица 26.

Ожидаемый результат	Код
Сообщение: «Нет изменений для сохранения», форма остаётся открытой	<pre> public void TC_04_AddRowAfterClear() { // Подготовка var form = new Form2(); form.dataGridView.Rows.Clear(); // Очистка таблицы AddNewStudentRow(); // Добавляем новую строку // Выполнение form.buttonSave_Click(null, EventArgs.Empty); // Проверка Assert.IsFalse(form.IsClosed, "Форма должна остаться открытой"); Assert.AreEqual("Не удалось сохранить данные. Таблица пуста", form.lastMessage, "Неверное сообщение об ошибке"); } </pre>

12.2. Функциональное тестирование

Для разработки тест-кейсов использовалась методика тест-дизайна эквивалентное разбиение. Методика выполняет следующие действия:

- Определение требований.
- Разделение на классы эквивалентности.
- Выбор представителей.
- Проверка на соответствие.

Группа: Группа А, Предмет: Математический анализ, Дата: 16.05.

ФИО	Оценка
Алексей Смирнов	-
Дмитрий Соколов	-
Иван Иванов	-
Мария Петрова	-
Ольга Кузнецова	-

Сохранить Назад

Рис. 21. Тестируемое поле ввода.

Требование:

Необходимо обязательное поле для оценок при выставлении оценок преподавателем. Входные данные должны быть целым числом от 2 до 5.

Таблица 27.

Классы эквивалентности	Представители классов	Тест-кейсы
Класс 1: Корректные оценки Класс 1.1: Числа в диапазоне от 2 до 5	Класс 1: Класс 1.1: «2» Класс 1.2: «3» Класс 1.3: «4» Класс 1.4: «5»	Тест-кейс 1: «4»
Класс 2: Некорректные оценки Класс 2.1: Значения вне диапазона	Класс 2: Класс 2.1: «1»	Тест-кейс 2: «1»
Класс 3: Недопустимые значения Класс 3.1: Буквы Класс 3.2: Спецсимволы Класс 3.3: Пустое значение	Класс 3: Класс 3.1: «А» Класс 3.2: «@» Класс 3.3: «»	Тест-кейс 3: «А» Тест-кейс 4: «»

Анализ граничных значений

Рис. 21. Тестируемое поле ввода.

Требование:

Поле оценок должно быть целочисленным в диапазоне от 2 до 5.

Таблица 28.

Классы эквивалентности	Представители классов	Тест-кейсы
Класс 1: Корректные значения	Класс 1: от 2 до 5	Тест-кейс 1: минимально допустимое значение «2» Тест-кейс 2: максимально допустимое значение «5»
Класс 2: Некорректные значения ниже допустимого минимума	Класс 2: от $-\infty$ до 1	Тест-кейс 3: значения ближайшие к границам «1», «3», «4», «6»
Класс 3: Некорректные значения выше допустимого минимума	Класс 3: от 6 до ∞	

Результаты тестирования

Успешные тест-кейсы: TC_01, TC_05, TC_06.

Неуспешные тест-кейсы: TC_02, TC_03, TC_04, TC_07, TC_08 (ожидаемые ошибки возникли).

13. Заключение

В ходе работы была разработана система учета успеваемости студентов, включающая функции выставления оценок, анализа успеваемости и формирования отчетов.

Система соответствует требованиям по автоматизации процессов, обеспечивает взаимодействие с базой данных PostgreSQL и работает через графический интерфейс на C#.

Проведен анализ аналогов, составлен план с использованием методов эквивалентного разбиения и анализа граничных значений, оценена трудоемкость этапов.

Реализованная система прошла тестирование, включая проверку корректности данных, обработку ошибок и навигации между формами.

Выявленные дефекты устранены, рекомендации по дальнейшему развитию включают экспорт отчетов в PDF и расширение функционала для работы с несколькими учебными годами. Разработанная система готова к внедрению в образовательные учреждения для повышения эффективности управления успеваемостью.

14. Список использованных источников

1. ГОСТ Р 56920—2016. Системная и программная инженерия. Тестирование программного обеспечения. Часть 1. Понятия и определения.
2. ГОСТ Р 56922— 2016/ISO/IEC/IEEE 29119- 3:2013 Системная и программная инженерия Тестирование программного обеспечения. Часть 3 Документация тестирования
3. РД IDEF0-2000 Методология функционального моделирования IDEF0
4. Шанченко, Н. И. Оценка трудоемкости разработки программного продукта: методические указания / Н. И. Шанченко. – Ульяновск: УлГТУ, 2015 – 40 с.
<http://lib.ulstu.ru/venec/2015/69.pdf>
5. Конструирование программного обеспечения: учебное пособие / составитель А. А. Романов. – Ульяновск: УлГТУ, 2016 – 126 с. <http://lib.ulstu.ru/venec/disk/2017/30.pdf>